

KillScore

The SaaS release gate that scores whether your AI-written tests can actually catch a bug.

`killscore.hajin.xyz`

The number you gate on cannot see this.

We flipped `>` to `>=` in a payment function. Every one of the 40 AI-written tests still passed. Coverage did not move.

LINE COVERAGE

100%

40 tests green

MUTATION SCORE

63%

10 of 27 planted bugs unnoticed

AI-written tests fail silently.

A test derived from the implementation cannot disagree with the implementation. If the code has a bug, the test records that bug as the expected value and reports green.

- About a quarter of merged commits in 2026 carry AI traces. The agent writes the tests too.
- Coverage measures whether a line ran. It cannot measure whether a test could have failed.
- The only way to find out is to break the code on purpose and see whether anything screams.

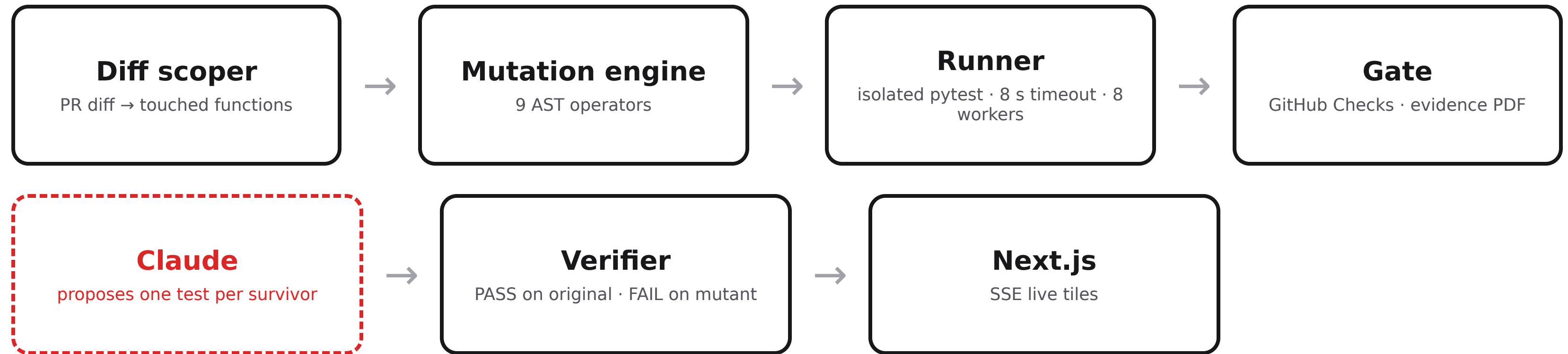
The person who presses merge.

One QA lead or release engineer in a 20 to 200 person engineering org that has adopted coding agents. Every week they approve a release on a single number.

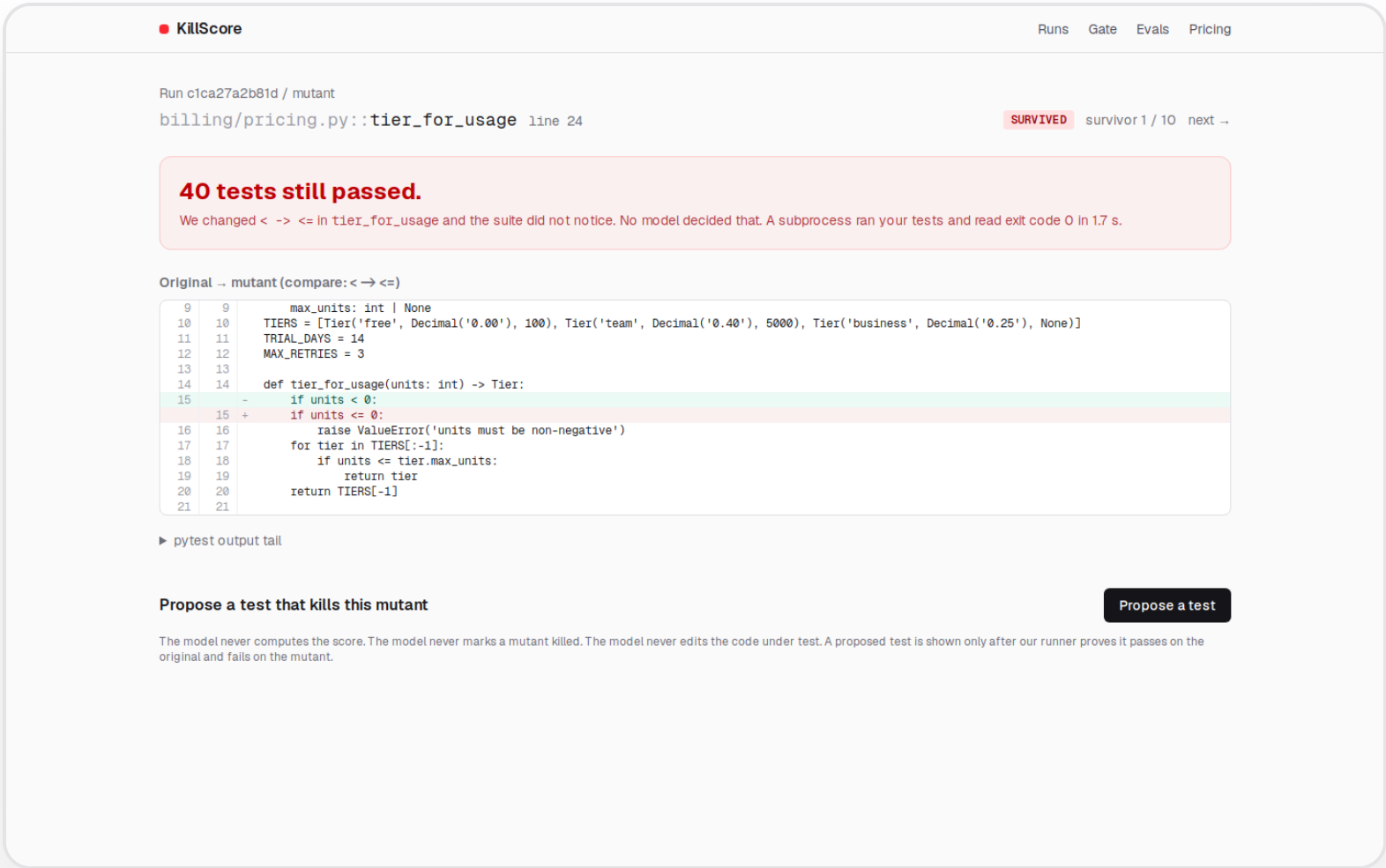
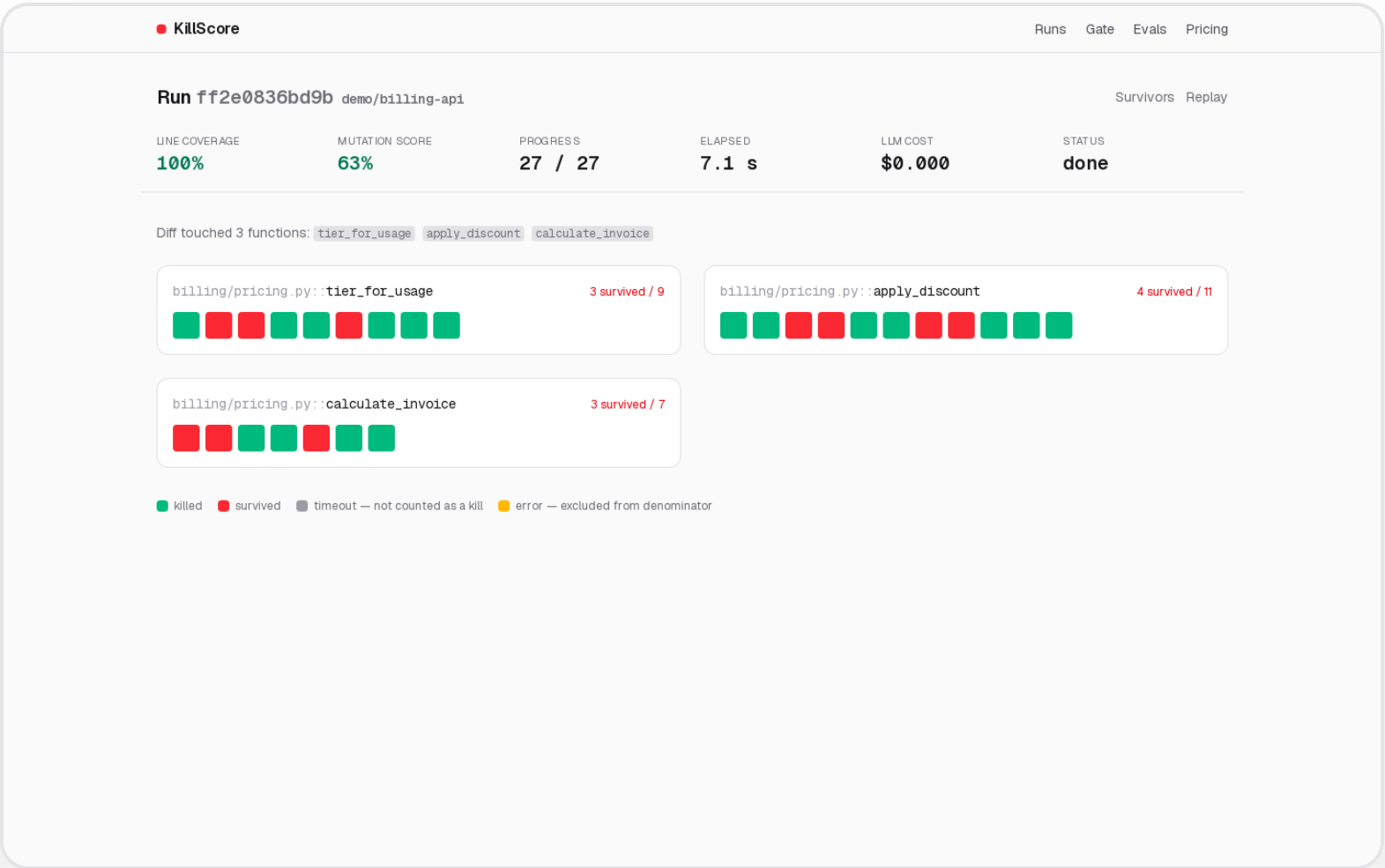


Codecov and SonarQube measure whether code ran.
We measure whether tests can disprove it.

Six components. One model call.



Diff scoping turns hours of full-repo mutation into seconds, which is what makes this a pull-request gate instead of a CLI. Timeouts stay in the denominator and never count as kills.



Left: tiles flip green, red or grey as each mutant's verdict lands. Right: a survivor, the exact diff, and the proposal panel.

Reviewer, not author.

- Claude Sonnet via OpenRouter, strict JSON schema output.
- Two-step verification: the proposed test must pass on the original and fail on the mutant before anyone sees it.
- A proposal citing a mutant id not in the run is rejected before execution.
- Every call logged: tokens, cost, latency. About \$0.015 per proposal.



The model never computes the score.
The model never marks a mutant killed.
The model never edits the code under test.
A test is shown only after our runner proves it.

On the demo repo: 10 proposals, 9 verified kills, 1 rejected because it failed on the original code. The rejection is the product working.

20 planted bugs. Coverage never moved.

METRIC	AI-WRITTEN SUITE (BASELINE)	LLM-ONLY REVIEWER (ABLATION)	KILLSCORE	N
Planted bugs detected	6 / 20	12 / 20	19 / 20	20
Mutation score	75%	—	75% → 97% after adding verified tests	101 mutants
Line coverage	100%	—	100%	—
Proposed tests verified	—	—	92% (22 / 24)	24
Tests shown without proof	—	—	0 (structurally impossible)	—
p50 / p95 per run	—	47.0s / 116.9s	2.5s / 4.9s	20
LLM cost per run	—	\$0.055	\$0.000 scoring · \$0.018 per proposed test	20

Twenty copies of a billing module, one planted bug each, AI-written suite at 100% line coverage. Regenerate with `make eval`. Live at /evals.

Priced per repo, not per seat.

Free

\$0

- Public repos
- 200 mutants per run
- No merge gate

Team

\$99 / month

- 5 private repos
- PR merge gate
- Slack alerts

Business

\$399 / month

- Unlimited repos
- Evidence PDF · SSO
- Self-hosted runner

Ship it to the merge button.

- GitHub App with the real Checks API
- JavaScript and TypeScript mutation operators
- Self-hosted runners with network-isolated containers for private code
- Survivor clustering so a reviewer sees five themes, not fifty tiles

killscore.hajin.xyz · github.com/SSALMUK-COMPOSE-COFFEE/DEVPOST_2026_AI_BUILDERS_HACKATHON